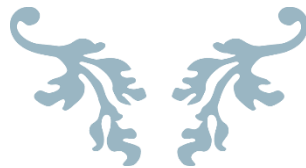




LEVERAGING BIG DATA FOR CLEARPATH PRIMARY CARE

A 30-DAY HOSPITAL READMISSION PIPELINE

Charmy Raj



JUNE 7, 2026

ISM-6362 FINAL PROJECT - KWIATKOWSKI, ANTHONY
SEATTLE PACIFIC UNIVERSITY

Table of Contents

Overview	1
Scope and Design	2
How the data moves	2
What each piece does	3
How I cleaned the data	3
Dashboard	4
Machine Learning Model.....	5
How I built it	5
How well it works and why it matters.....	6
Conclusion.....	6

Overview

ClearPath Primary Care is a primary care clinic network that looks after about 70,000 chronic-disease patients across five clinics. The problem I built this project around is hospital readmissions. When one of ClearPath's patients gets admitted to a hospital and then comes back within 30 days, the clinic takes a financial hit from Medicare's readmission penalties, and it usually means the patient's care wasn't coordinated well enough after they went home. Across the whole population, roughly 9% of discharged patients end up back in the hospital within 30 days. ClearPath only has a handful of care managers, so they can't go through every patient chart by hand. They need a system that tells them who is getting worse right now, where readmissions are piling up, and which specific patients are most likely to come back so they can reach out before it happens.

My solution uses the clinic's data in three different ways.

The first is a **streaming** use case for real-time vitals alerting. Patient check-ins and home monitors send a constant stream of vitals like heart rate, blood pressure, oxygen, and temperature. I process these as they come in using a five-minute window with a watermark for late events, and I raise an alert any time a patient crosses a clinical threshold, so a care manager can step in while it's happening instead of finding out later.

The second is a **batch** use case for nightly KPIs. Every night the system rolls up the full encounter history into three metrics that show the care team where to focus: 30-day readmission rate by diagnosis group, average length of stay by admission type, and 30-day readmission rate by how many times a patient has been hospitalized before. These are reported on a nightly batch schedule so the team starts each morning with a current picture.

The third is a **machine learning** use case. I trained a model that gives each patient a probability of being readmitted within 30 days, using their demographics, diagnosis, prior visits, and labs. The output is a ranked risk list, which lets the clinic put its limited outreach where it matters most.

I had originally planned to use the MIMIC-IV dataset for this project, but I couldn't get it to download. The full dataset wouldn't come through, and the demo version that was available only had 100 records, which is far too small to build a real big-data pipeline or train a model on. So I switched to the Diabetes 130-US Hospitals (1999-2008) dataset instead. It has 101,766 diabetic encounters with demographics, diagnoses, prior-visit counts, labs, and a 30-day readmission label, and since diabetes is mostly managed in primary care, this population still fit the ClearPath story well.

Scope and Design

How the data moves

I set the pipeline up as a Bronze / Silver / Gold architecture on Amazon S3, with Spark on Databricks doing the processing and Power BI on the front end. The data takes two parallel paths that only meet at the business level: a real-time path for the vitals stream and a historical path for the batch metrics and the model.

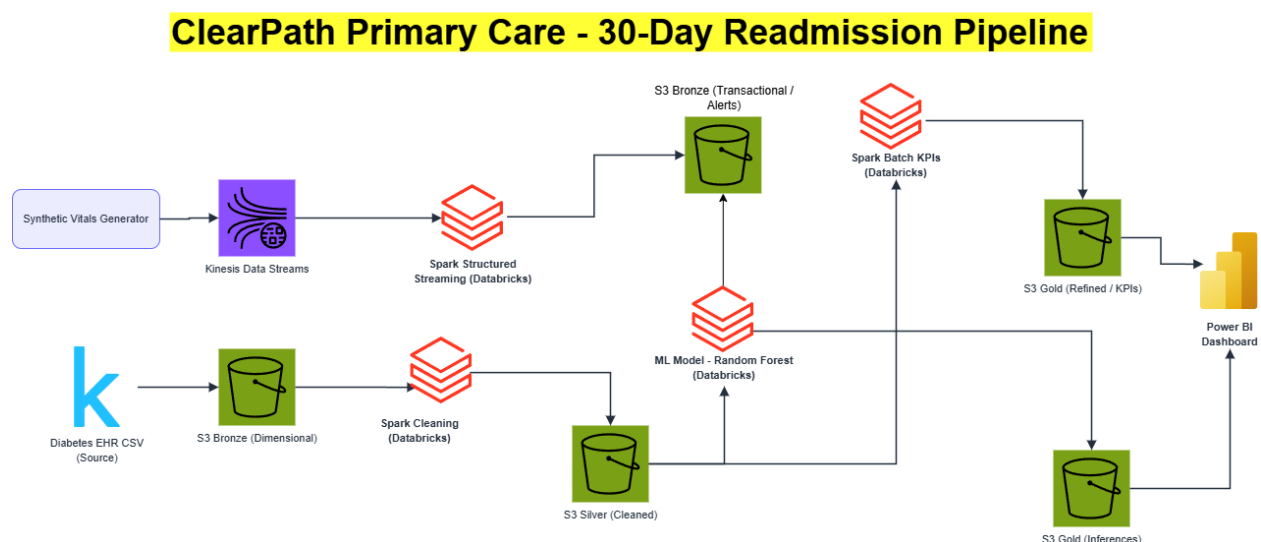


FIGURE 1: Architecture diagram

On the streaming path, synthetic vitals events get generated and dropped as JSON into the S3 Bronze (Transactional) bucket, which stands in for a Kinesis stream. Spark Structured Streaming reads that stream, groups the events into five-minute windows per patient with a ten-minute watermark, and tags anyone who crosses a threshold as TACHYCARDIA, HYPERTENSIVE, LOW_OXYGEN, or FEVER.

On the historical path, the diabetes data loads from the S3 Bronze (Dimensional) bucket and gets cleaned (I describe that below). The cleaned data goes into S3 Silver. From there, two things run off it: a batch job that builds the three KPIs and writes them to S3 Gold (Refined), and the machine learning job that trains the model and writes each patient's risk score to S3 Gold (Inferences). Power BI reads the Refined and Inferences buckets to build the dashboard.

What each piece does

The **Synthetic Vitals Generator** produces the live event stream. **Kinesis Data Streams** is the ingestion layer that buffers those events. **Spark Structured Streaming on Databricks** runs the windowed alert query. **S3 Bronze** is the raw landing zone - vitals on the Transactional side, the diabetes encounters on the Dimensional side. **S3 Silver** holds the cleaned, deduplicated, labeled data. **Spark Batch on Databricks** computes the nightly KPIs. **S3 Gold** holds the finished outputs - the KPI tables in Refined and the model's risk scores in Inferences. **Power BI** reads Gold and presents the dashboard to care managers.

I kept the vitals stream and the diabetes data separate on purpose, because they're different shapes. The vitals are timestamped events you can window over time, and the diabetes data is one summary row per patient that carries the readmission label, so each one only fits the use case it was built for.

How I cleaned the data

I started with the raw `diabetic_data.csv`, which had 101,766 rows. The missing values weren't blank in this file - they were stored as a question mark - so the first thing I did was convert those to proper nulls in the string columns so Spark would treat them as missing. Then I looked at which columns were usable. Weight was missing for about 97% of patients and payer code for around 40%, so I dropped both. A lot of the medication columns held the same value for nearly every patient, so I dropped those too since they had no information for the model.

After that I removed rows that didn't make sense for a readmission study. A few patients had an unknown gender, so those came out. More importantly, I removed any patient who had died or been discharged to hospice, because you can't readmit someone who passed away and keeping them in would distort the label. The last cleaning step was making sure each patient only showed up once. Some patients had several visits, and if the same person ended up in both my training and test data the model would basically be cheating, so I kept only each patient's first encounter. That left me with 69,987 patients, which is almost exactly the 69,984 the original dataset paper reported - so I knew my cleaning matched their method. The final readmission rate came out to about 9%.

Dataset: <https://www.kaggle.com/datasets/brandao/diabetes>

Dashboard

The dashboard shows the three nightly KPIs to care managers, with cards at the top summarizing the population.

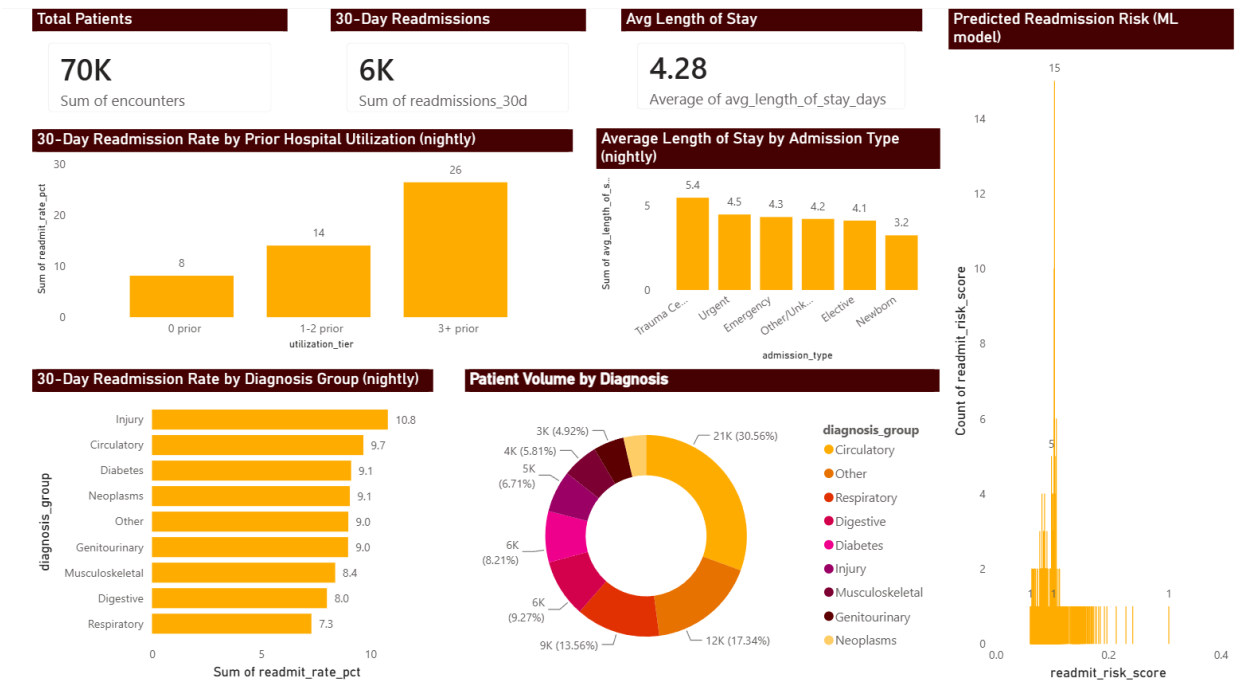


FIGURE 2: Power BI dashboard screenshot

KPI 1 - 30-Day Readmission Rate by Diagnosis Group, reported nightly, shows which conditions drive the most readmissions. Injury (10.8%) and circulatory disease (9.7%) are highest; respiratory is lowest at 7.3%.

KPI 2 - Average Length of Stay by Admission Type, reported nightly, uses length of stay as a stand-in for how severe and costly a case is. Trauma Center admissions average 5.4 days down to 3.2 days for newborns.

KPI 3 - 30-Day Readmission Rate by Prior Hospital Utilization, reported nightly, is the clearest signal for who to target. The rate climbs from 8.1% for patients with no prior inpatient visits, to 14.0% for one or two, to 26.5% for three or more. The patients in that top tier are the ones the clinic should reach out to first.

Along the top of the dashboard I put three summary cards so anyone looking at it gets the scale of the problem right away: about 70,000 patients in total, roughly 6,000 of them readmitted within 30 days, and an average length of stay of 4.28 days. The charts below them break that down.

I added a Patient Volume by Diagnosis chart next to the readmission-rate-by-diagnosis chart on purpose, because the two answer different questions. The rate chart shows that Injury and Circulatory patients are the most likely to be readmitted, but the volume chart

shows that Circulatory is by far the largest group, at about 31% of all patients. So even though Injury has the higher rate, Circulatory patients probably account for more total readmissions simply because there are so many of them. Looking at rate and volume together keeps the clinic from chasing a high percentage that only applies to a small handful of patients.

I also included a chart of the model's predicted risk scores so the dashboard connects back to the machine learning side. Most patients sit at a low risk around 0.1, with a smaller tail stretching toward 0.3 and above. Those higher-risk patients on the right are the ones the model is flagging, and they are who the care managers would reach out to first.

Put together, the dashboard moves from the overall numbers at the top, down to where readmissions concentrate by patient history, admission type, and diagnosis, and finally to the model's per-patient risk score, so a care manager can go from the big picture to a specific list of who to call first.

Machine Learning Model

How I built it

I was predicting whether a patient would be readmitted within 30 days, so I turned the readmitted field into a simple 1/0 column. For features I picked the things I figured would matter - age group, diagnosis group, admission type, A1C result, length of stay, number of medications and lab procedures, and how many times the patient had been to the hospital before. I split the data 80/20 so I could test on patients the model had never seen, which gave me about 56,000 to train on and 14,000 to test.

I used a Random Forest because it handles a mix of categorical and numeric features well, and I built it as a Spark ML pipeline so the encoding and the model run together in one flow. I also trained a Logistic Regression as a baseline to compare against.



```
▶ ✓ Yesterday (25s) 10

# STEP 3: Evaluate with AUC-ROC (+ Logistic Regression baseline)
rf_predictions = rf_model.transform(test_data)

evaluator = BinaryClassificationEvaluator(labelCol="label", metricName="areaUnderROC")
rf_auc = evaluator.evaluate(rf_predictions)
print(f"Random Forest AUC-ROC: {rf_auc:.3f}")

lr = LogisticRegression(labelCol="label", featuresCol="features")
lr_model = Pipeline(stages=indexers + [encoder, assembler, lr]).fit(train_data)
lr_auc = evaluator.evaluate(lr_model.transform(test_data))
print(f"Logistic Regression AUC-ROC: {lr_auc:.3f}")

> rf_predictions: pyspark.sql.connect.dataframe.DataFrame
Random Forest AUC-ROC: 0.597
Logistic Regression AUC-ROC: 0.596
```

FIGURE 3: ML output - risk-score distribution/ the AUC printout

How well it works and why it matters

For scoring I used AUC-ROC instead of accuracy, and that was on purpose. Only about 9% of patients actually get readmitted, so a model that just guessed “no” every time would look 91% accurate while being useless. AUC tells you whether the model is ranking the risky patients above the safe ones, which is what actually matters here. I ended up around 0.60 - not huge, but better than a random 0.50, and readmission is genuinely hard to predict. When I looked at what the model leaned on most, it was the diagnosis group and how many prior hospital visits a patient had, which matched my batch KPI where patients with more past admissions had much higher readmission rates.

The real value of the model isn't perfect prediction, it's prioritization. Scoring the whole population is cheap and automatic, and even a model that's only moderately accurate but reliably surfaces the highest-risk group is far more useful than reviewing charts by hand or treating everyone the same. A single 30-day readmission is expensive and comes with a penalty, so catching even a portion of them pays for the system many times over. That's the cost/benefit side of big data - the decision to run the model is worth it well before the model is highly accurate.

Conclusion

I set out to help ClearPath Primary Care cut down on 30-day readmissions, and I built an end-to-end pipeline to do it. The system takes in real-time vitals and raises alerts, rolls up historical encounters into nightly KPIs, and predicts each patient's readmission risk with a machine learning model, all organized in a Bronze/Silver/Gold setup on S3 and Spark/Databricks and shown through a Power BI dashboard.

The cleaned dataset of 69,987 patients had an 8.98% readmission rate. The KPIs showed that readmission risk piles up among patients with prior admissions - going from 8% to 26% as prior visits increase - and varies by diagnosis. The Random Forest landed at an AUC of about 0.60, with prior utilization and diagnosis group as its main drivers, which lines up with what the KPIs already showed.

The hardest part was getting a cloud architecture working within the platform limits. I represented the AWS streaming and storage layers with Databricks Volumes and Structured Streaming, and I ran into things like the serverless session dropping my variables and the streaming query needing an explicit checkpoint location before it would run, both of which took some trial and error to sort out. I also had to be careful with the cleaning to avoid leaking patients between train and test and to keep the label honest. If I had more time, the things I'd add are wiring the streaming alerts to actually send notifications, handling the class imbalance with class weighting or resampling so the model catches more of the readmitted patients, and bringing in more clinical features. Even as it stands, the project runs the whole big-data lifecycle - ingestion, cleaning,

streaming, batch aggregation, machine learning, and visualization - on a realistic healthcare problem.